

Sliding Puzzle Project — Development Log

Day 3 — Tuesday, August 4, 2026

Goal for the project

Build four Python puzzle projects (a playable sliding tile puzzle, a solver for it, a playable 3D Rubik's cube, and a solver for that) as a learning exercise. The user writes all code themselves; guidance is conceptual, not code handed over directly.

Today's focus

Stage 2 of the sliding puzzle: core movement logic. Writing and debugging `find_blank()`, `legal_moves()`, `apply_move()`, and `is_solved()` in a new file, `movement_logic.py`, keeping `board.py` focused on board representation/display only.

What happened

- Decided to split movement logic out of `board.py` into a new file, `movement_logic.py`, which imports `board.py`. Reasoning: `board.py` handles structure/display, `movement_logic.py` handles game rules/behaviour — different responsibilities, and keeps `board.py` reusable by both the playable game and the future solver project.
- Wrote and debugged `find_blank(dictionary)` — loops over the board dictionary with `.items()` to find the key whose value is `"_"`. Initial version looped over the dictionary directly (missing `.items()`), which silently unpacked each `(row,col)` key into the two loop variables instead of giving key+value pairs — no error was raised, but the blank was never actually found.
- Wrote and debugged `legal_moves(dictionary)` — finds the blank's position via `find_blank`, then checks all four orthogonal neighbours `((r0±1,c0)` and `(r0,c0±1))` against the dictionary's own keys, using the fact that only valid `(row,col)` pairs exist as keys at all (so no separate boundary/range check was needed — invalid neighbours simply aren't in the dictionary). Two real bugs caught and fixed along the way:
- An early version returned inside the loop on the first match, which exits the function immediately — meaning only one legal move was ever found even when several existed. Fixed by collecting matches into a list with `.append()` and returning only after the loop finished.
- An early version of the neighbour check chained comparisons as `(r,c) == (r0+1,c0) or (r0-1,c0) or ...` — only the first comparison actually used `==`; the remaining tuples were evaluated as plain (always truthy) values, so the condition was always `True` regardless of position. Fixed by writing out a full `(r,c) == ...` comparison for each of the four neighbours.
- Wrote and confirmed `apply_move(dictionary, move)` — checks the requested move is in `legal_moves()` first, then swaps the tile at the target position with the blank; returns `True/False` so calling code (a future game loop) can branch on success, with a separate `print()` used for human-readable failure messages so the return value stays clean for code to check.
- Split `board.py`'s `create_solved_board()` into three functions: `choose_size()` (menu + input only), `build_solved_board(size)` (builds a flat tile list for a given size, no input prompt), and `create_solved_board()` (now just calls both in sequence). This was needed so `is_solved()` could build a comparison board internally without re-prompting the user for a size every time it checks for a win.
- Wrote and confirmed `is_solved(dictionary)` — derives tile count via `len(dictionary)` (confirmed dictionaries support `len()` the same as lists), maps that to the corresponding size string, builds a solved comparison

dictionary via `build_solved_board()` + `create_grid_dictionary()`, and compares the two dictionaries directly with `==`.

- All four Stage 2 functions tested end-to-end on a 3x3 board: `find_blank` correctly returned the blank's position; `legal_moves` correctly returned both valid neighbours from a corner position; `apply_move` correctly performed a swap and correctly rejected an illegal move; `is_solved` correctly returned `True` on a fresh board and `False` after a single move.

Concepts & Lessons Learned (study notes)

Looping over a dictionary: keys vs. keys+values

Looping directly over a dictionary (for `x` in `dictionary`:) only gives keys, not values. Three loop styles exist: for key in dictionary: / `dictionary.keys()` (keys only), `dictionary.values()` (values only), `dictionary.items()` (key and value together, unpacked as a pair). This can fail silently rather than crash: if a dictionary's keys happen to be tuples of the right length, for `a, b` in `dictionary`: still runs without error, but `a` and `b` end up holding parts of the key, never the actual value — so a condition checking the value never matches. Use `.items()` whenever both key and value are needed.

or does not distribute across a comparison

Writing `x == a or b or c` is not the same as `x == a or x == b or x == c`. Python evaluates `==` before `or`, so only the first comparison is real; the remaining values (`b`, `c`) are evaluated on their own as plain values. Since most non-empty/non-zero values — including any non-empty tuple — are truthy in Python, a chain like this can end up always `True` regardless of `x`, which is a silent logic bug, not an error. Each comparison must be written out in full with its own `==`.

return inside a loop exits immediately, not just once

A `return` statement inside a `for` loop ends the function the instant it executes, on the very first match — the loop never continues to check remaining items, so any other matches are never even discovered (not overwritten or discarded — simply never reached). To collect every match in a loop, use a list created before the loop, `.append()` inside the condition, and `return` only after the loop has fully finished, indented back out to loop level.

Functions that change state should report success or failure

A function that modifies something (like swapping values on a board) and has a real failure case (an illegal move) should tell its caller whether it actually succeeded, rather than returning nothing meaningful either way. Chosen approach: `return True/False` for code to branch on cleanly, with a separate `print()` statement reserved for human-readable messages — keeping what the function returns (for other code) and what it prints (for a person watching the terminal) as two distinct concerns.

Restructuring a function to remove a side effect (splitting out `input()`)

`create_solved_board()` originally combined two jobs: asking the user for a size via `input()`, and building the corresponding board. This became a problem once `is_solved()` needed to build a comparison board internally — it would have re-prompted the user for input every time a solved-check ran. Fix: split into `choose_size()` (input only) and `build_solved_board(size)` (pure logic, no input), with the original function reduced to calling both in sequence. General lesson: a function that mixes user interaction with pure computation is harder to reuse elsewhere — separating the two lets other code call just the computation part.

`len()` works on dictionaries, not just lists/strings

`len(dictionary)` returns the number of key-value pairs, the same way `len(list)` returns the number of items. This let `num_of_columns()` and the tile-count logic in `is_solved()` work directly off a dictionary without needing the original flat list or size string passed in separately.

A local variable can share a function's name, but it's risky style

Python allows a variable inside a function to have the same name as the function itself (e.g. `legal_moves = []` inside `def legal_moves():`) — they exist in different scopes, so no error occurs. The real risk: once the local variable is assigned, that name refers to the variable for the rest of the function's execution, which would break a recursive call

to the function from within itself. Not an issue here since neither function calls itself, but worth naming variables distinctly from their enclosing function for clarity.

Status at end of day

Stage 2 (core movement logic) is complete: `find_blank()`, `legal_moves()`, `apply_move()`, and `is_solved()` are all written, debugged, and confirmed working via manual terminal tests on a 3x3 board. `board.py` was refactored (`create_solved_board()` split into `choose_size()/build_solved_board()`) without breaking existing behaviour. Movement logic now lives in its own file, `movement_logic.py`, importing `board.py`.

Next up

Stage 3 — shuffling: starting from a solved board and applying a couple hundred random legal moves (skipping the immediate reverse of the previous move) to produce a solvable, well-mixed starting board, rather than placing tiles randomly.